# Deep neural networks for classification and detection

**Matej Popovski and Sean Chang**
BMI / CS 771 - Assignment 2 - Fall 2026

## Scope and measured outcomes

This report covers all required classification and object-detection tasks [1]. SceneCNN achieves 48.74% top-1 accuracy on MiniPlaces, improving on SimpleNet by 5.31 percentage points. The independently trained FCOS detector achieves 60.52% AP50 on VOC2007 test, close to the instructor checkpoint at 60.69%. Optional Swin and COCO experiments are not claimed.

| Classifier | Top-1 | Top-5 | Evaluation images |
|---|---|---|---|
| SimpleNet | 43.43% | 72.49% | 10,000 |
| SceneCNN | 48.74% | 77.60% | 10,000 |
| Local/global ViT | 45.12% | 74.61% | 10,000 |
| ImageNet ResNet-18 | 56.45% | 83.81% | 10,000 |

| Detector | AP50 | AP50:95 | Evaluation images |
|---|---|---|---|
| Instructor reference | 60.69% | 32.80% | 4,952 |
| Independently trained FCOS | 60.52% | 32.42% | 4,952 |

## Team contributions

**Matej Popovski:** Classification component: custom CNN, SimpleNet/ResNet-18 comparison, multi-head attention and ViT experiments; repository setup, artifact verification and combined report/submission integration (6 required points).

**Sean Chang:** TransformerBlock review and FCOS inference/loss component; Colab T4 reference evaluation, detector training/evaluation, saved artifacts and detection analysis (6 required points).

AI tools assisted with implementation, experiments, verification, analysis and writing.

The split assigns six required points to each member. The final package combines both components, their trained checkpoints, logs, configurations, tests and measured results.

# 1. CNN design and classification protocol

MiniPlaces supplies 100,000 training images and 10,000 labeled validation images across 100 classes. Only the training split updates weights. SimpleNet, SceneCNN and ViT start from random initialization; the designated ResNet-18 comparison alone uses ImageNet pretraining. All reported results use the final scheduled checkpoint and the entire validation set.

Seed 771 is used throughout. Training applies a random resized 128 x 128 crop, horizontal flip, color jitter 0.15, and rotation up to 15 degrees. Validation resizes the short side to 144, then center-crops 128 x 128, with no flip averaging. Both use ImageNet channel normalization. Cross-entropy is minimized; the learning-rate scheduler steps after each optimizer update.

| SceneCNN stage | Structure | Output (channels x height x width) |
|---|---|---|
| Stem | 3 x 3 convolution, stride 2; batch norm; ReLU | 32 x 64 x 64 |
| Residual stage 1 | Two blocks, channels 32; first stride 2 | 32 x 32 x 32 |
| Residual stage 2 | Two blocks, channels 64; first stride 2 | 64 x 16 x 16 |
| Residual stage 3 | Two blocks, channels 128; first stride 2 | 128 x 8 x 8 |
| Residual stage 4 | Two blocks, channels 256; first stride 2 | 256 x 4 x 4 |
| Classifier | Global mean pool; dropout 0.2; linear 256 to 100 | 100 logits |

Each residual block has two 3 x 3 convolutions with batch normalization, a ReLU between them, and a ReLU after the shortcut addition. A 1 x 1 projection and batch normalization align changing dimensions. The last normalization scale in each residual branch starts at zero. Residual connections [2] provide a short gradient path; narrow early stages and downsampling limit computation. Global pooling and dropout control the classifier size and regularize its representation.

| Model | Optimizer | Schedule | Batch / device |
|---|---|---|---|
| SimpleNet | SGD, lr 0.1 | 1 warmup + 40 cosine | 128 / MPS |
| SceneCNN | SGD, lr 0.1 | 1 warmup + 40 cosine | 128 / MPS |
| ResNet-18 | SGD, lr 0.01 | 1 warmup + 15 cosine | 128 / MPS |

All three SGD runs use momentum 0.9 and weight decay 0.0001. The supplied SimpleNet uses convolutional groups with bottlenecks, ReLUs, max pooling and global average pooling. For ResNet-18, a new 100-class head replaces the ImageNet head and all parameters are fine-tuned. Pretraining and the shorter schedule make it a transfer-learning reference, rather than an architecture-only comparison.

## 2. Attention, TransformerBlock and ViT

Multi-head attention [3] jointly projects tokens into queries Q, keys K and values V, splits channels into heads, and computes softmax($QK^T$/sqrt(d))V, where d is the channel dimension per head. Softmax runs across keys. Head outputs are concatenated and passed through an output projection. Tests compare values and input gradients with PyTorch scaled-dot-product attention.

| Component | Chosen ViT design |
|---|---|
| Patch embedding | 8 x 8 nonoverlapping patches; 16 x 16 = 256 tokens; 192 channels |
| Position / depth | Learned absolute position embeddings; six blocks; four heads |
| Attention layout | 4 x 4 local windows in blocks 0, 1, 3, 4; global attention in blocks 2, 5 |
| MLP / output | Expansion ratio 4; GELU; final normalization, mean token pooling and 100-class head |
| Regularization | Stochastic-depth rates increase linearly from 0 to 0.1 |

The design follows patch-token processing [4]. A local window contains 16 tokens, compared with 256 globally, reducing the quadratic attention-pair count by a factor of 16 in local blocks. Projection and MLP costs remain. Global blocks exchange information across windows; learned positional embeddings interpolate if the input grid changes.

## Pre-LN residual block and local attention

For X shaped (batch, height, width, channels), the block computes Y = X + DropPath(Attention(LN1(X))) and Z = Y + DropPath(MLP(LN2(Y))). Normalization precedes both learned branches (Pre-LN [5]). Positive window size selects local attention; zero selects global attention. Spatial padding follows LN1, and unpartitioning removes it before the residual addition. The starter's zero-token padding convention has no attention mask; this behavior and parameter shapes were preserved.

Stochastic depth [6] samples one keep/drop decision per original image and residual branch, broadcasts over spatial positions and channels, and scales kept branches by the inverse keep probability. It runs after window reassembly, so windows from an image share the same decision. The two branches draw independent masks. Evaluation disables dropping. Tests cover a window-loop oracle, rectangular padded inputs, zero cross-window gradients, residual identity and image-level masks.

## ViT training

AdamW uses learning rate 0.001, weight decay 0.05, batch size 256, and 1 warmup plus 40 cosine epochs on a Colab Tesla T4. Training uses mixed FP16 and evaluation FP32. No ImageNet initialization is used. Its batch differs from the CNN runs, so the number of optimizer updates per epoch also differs. No extensive hyperparameter search or multi-seed comparison is claimed.

# 3. Classifier learning curves

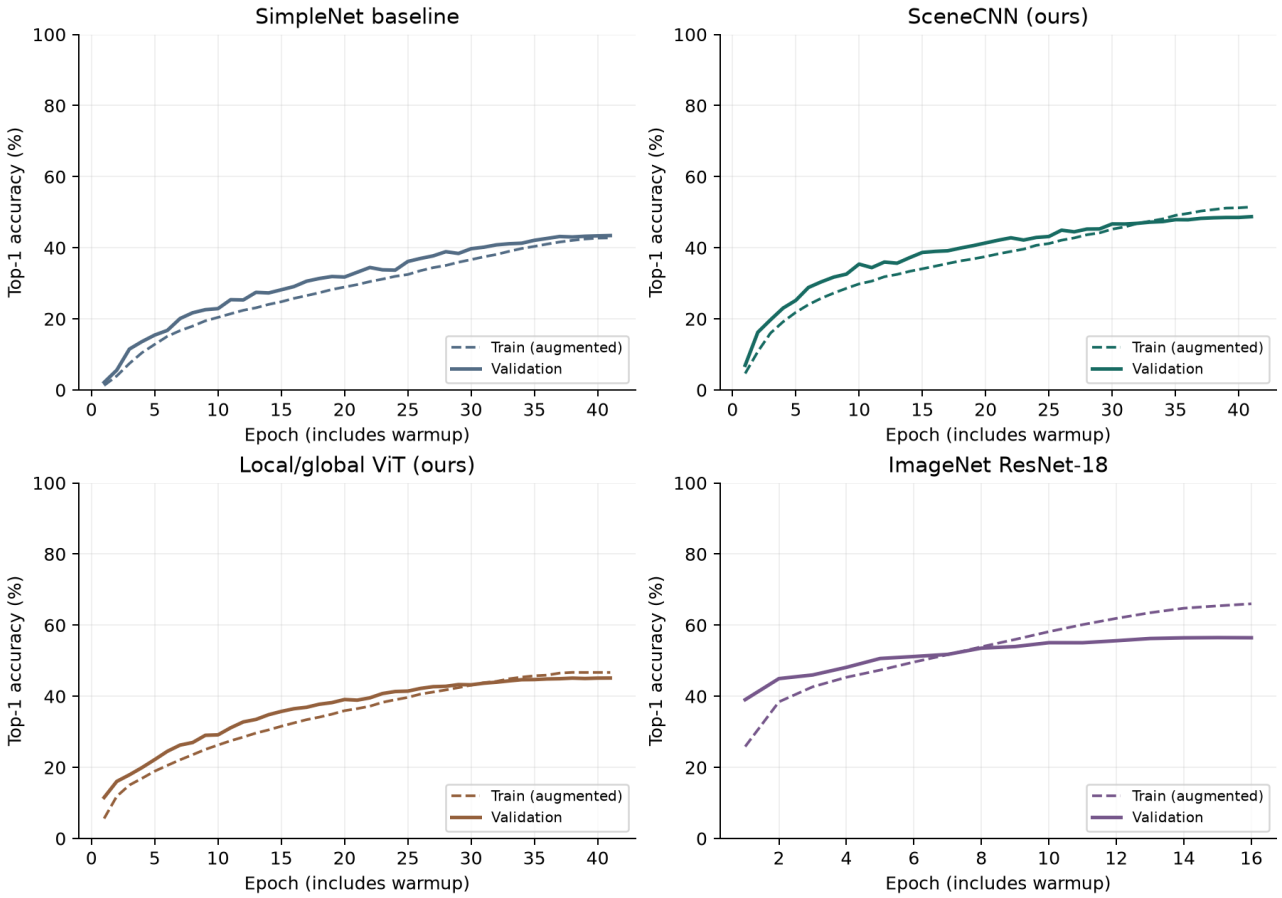



Figure 1. Full-epoch training top-1 accuracy (dashed) and validation top-1 (solid). Epochs include warmup.

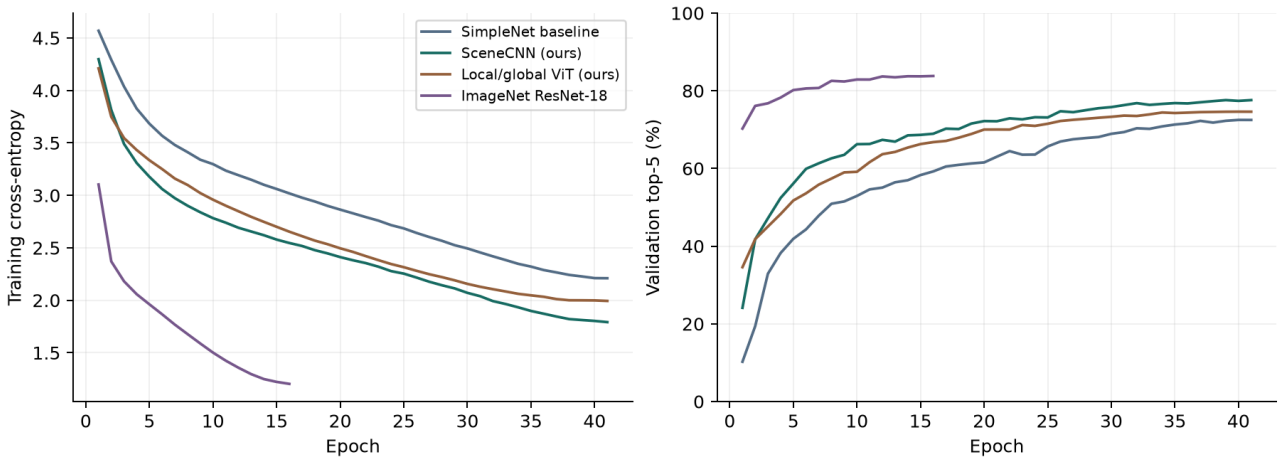



Figure 2. Training cross-entropy and validation top-5 accuracy. Training augmentation and dropout can make training accuracy lower than validation accuracy.

# 4. Classification analysis and efficiency

| Model | Params (M) | MACs (M) | Top-1 | Top-5 |
|---|---|---|---|---|
| SimpleNet | 0.37 | 160.2 | 43.43% | 72.49% |
| SceneCNN | 2.82 | 143.0 | 48.74% | 77.60% |
| Local/global ViT | 2.78 | 745.6 | 45.12% | 74.61% |
| ImageNet ResNet-18 | 11.23 | 592.2 | 56.45% | 83.81% |

MACs estimate one forward pass at 128 x 128, counting convolution/linear operations and both attention matrix products. Normalization, nonlinearities, softmax, pooling, biases and residual additions are excluded. This operation estimate is not a runtime measurement.

SceneCNN improves top-1 over SimpleNet by 5.31 percentage points, but trails fine-tuned ResNet-18 by 7.71 points. SceneCNN uses about one quarter of ResNet-18's parameters. Against SimpleNet, it has 7.73 times as many parameters but 10.7% fewer estimated MACs, because more capacity operates on downsampled maps.

ViT improves on SimpleNet by 1.69 points but trails SceneCNN by 3.62 points. Despite similar parameter counts, its estimated forward MACs are 5.21 times SceneCNN's, largely due to token projections and MLPs. For these measured configurations, SceneCNN gives the strongest scratch-trained accuracy and computation tradeoff. One seed and differing optimizers do not establish an architecture-wide conclusion.

| Model | Final train loss | Best val epoch / top-1 | Train min | Eval sec |
|---|---|---|---|---|
| SimpleNet | 2.210 | 41 / 43.43% | 48.3 | 14.8 |
| SceneCNN | 1.792 | 41 / 48.74% | 20.9 | 6.1 |
| Local/global ViT | 1.994 | 41 / 45.12% | 122.3 | 13.0 |
| ImageNet ResNet-18 | 1.204 | 15 / 56.49% | 21.7 | 7.5 |

Best validation epochs are descriptive; headline scores use final scheduled checkpoints. Training minutes sum training loops and exclude validation/setup. Evaluation seconds cover the full validation loader and inference.

**SimpleNet:** validation top-1 rises from 2.13% after warmup to 43.43% at the final epoch. The final training-minus-validation gap is -0.66 points.

**SceneCNN:** validation top-1 rises from 6.94% after warmup to 48.74% at the final epoch. The final training-minus-validation gap is +2.72 points.

**Local/global ViT:** validation top-1 rises from 11.53% after warmup to 45.12% at the final epoch. The final training-minus-validation gap is +1.58 points.

**ImageNet ResNet-18:** validation top-1 rises from 39.04% after warmup to 56.45% at the final epoch. The final training-minus-validation gap is +9.55 points.

SimpleNet, SceneCNN and ResNet-18 ran in FP32 on an Apple M3 Max (30 GPU cores, 36 GB unified memory); ViT ran on a Colab T4. Hardware, precision, batch size and caching differ, so timings are descriptive. Augmentation/dropout also confound training-versus-validation gaps; they are not isolated estimates of overfitting.

# 5. FCOS inference, targets and training

The assignment FCOS architecture [1, 7] uses ResNet-18 features at strides 8, 16 and 32, a 128-channel FPN, and extra levels at strides 64 and 128. Shared classification and regression towers each contain three convolution/GroupNorm/ReLU stages. Heads predict 20 class logits, nonnegative side distances and centerness. This implementation extends the starter; it does not call torchvision or another external FCOS detector.

## Inference

PointGenerator stores (y, x), while boxes use (x, y). Scores are sqrt(sigmoid(class) * sigmoid(centerness)). Thresholding and top-K selection operate on class-location pairs, allowing several class hypotheses per location. Left/top/right/bottom distances are multiplied by the level stride and decoded around the point. Boxes are clipped to resized bounds, and dimensions at most 0.01 pixel are rejected. Class-aware NMS runs across levels, followed by a per-image limit. Labels receive +1 for the supplied category IDs, and boxes rescale to original image dimensions.

Evaluation uses score threshold 0.1, NMS IoU 0.6, top 1,000 candidate class-location pairs per level and at most 100 detections per image. The short side is 480 with long side capped at 640. The report's visualizations use a separate display threshold of 0.5.

## Target assignment and multitask loss

A location must lie strictly inside a ground-truth box and its center region, clipped to the box. Center radius is 1.5 times the level stride. The largest pixel distance to the box sides must fall in that level's regression range: [0,64], [64,128], [128,256], [256,512], [512,1000]. The smallest-area eligible box resolves overlaps. Class targets are zero-based; unmatched points have all-zero class targets. Regression targets are side distances divided by stride. Assignment is vectorized over points and boxes, with a loop only over images.

The objective adds sigmoid focal classification (alpha 0.25, gamma 2) over all locations, unweighted GIoU regression over positives, and BCE-with-logits centerness over positives. The centerness target is sqrt(min(l,r)/max(l,r) * min(t,b)/max(t,b)). Each term divides by the starter's exponential moving average of positive counts (initial 100, momentum 0.9, minimum 1). Empty-positive batches retain classification loss and differentiable zero regression/centerness losses.

## Data and optimization

Only VOC2007 trainval (5,011 images) trains the detector; all 4,952 test images are reserved for evaluation. Image files, annotations, 20 categories and disjoint split filenames were checked. Training initializes the ResNet-18 backbone from ImageNet, with new FPN and heads; it never loads the instructor detector.

Colab T4 training uses seed 771, batch 32, SGD lr 0.02, momentum 0.9, weight decay 0.0001, 3 warmup plus 12 cosine epochs and mixed FP16. Short-side scale jitter is [288,320,352,384,416,448,480], maximum long side 640; horizontal flips update boxes. Backbone batch normalization is not frozen. Final evaluation uses FP32.

# 6. Detection results and efficiency

| Detector | AP50 | AP50:95 | AP75 | Images |
|---|---|---|---|---|
| Instructor reference | 60.69% | 32.80% | 31.88% | 4,952 |
| Trained FCOS | 60.52% | 32.42% | 31.52% | 4,952 |

Scores use the supplied COCO-format test annotations and pycocotools COCOeval, rather than the VOC 11-point AP formula. AP50 is the assignment's main metric; AP50:95 averages IoU thresholds from 0.50 through 0.95. The trained detector is 0.17 AP50 points below the reference. The instructor reference at 60.69% is close to the approximately 60.9% expected by the assignment.

| T4 evaluation | Batch | Model sec / img/s | Data-loop sec | Full eval sec |
|---|---|---|---|---|
| Reference | 8 | 102.66 / 48.24 | 126.99 | 177.70 |
| Trained FCOS | 32 | 83.60 / 59.24 | 108.22 | 158.02 |

Synchronized model time includes resize/normalize, backbone, heads, decode/NMS and box rescaling, including the cold first batch. Data-loop time adds loading, transfers, output conversion and sample visualization writes. Full evaluation adds prediction JSON and COCOeval. Both runs use Tesla T4 and PyTorch 2.11.0+cu130. Different batches and candidate counts prevent an intrinsic model-speed claim.

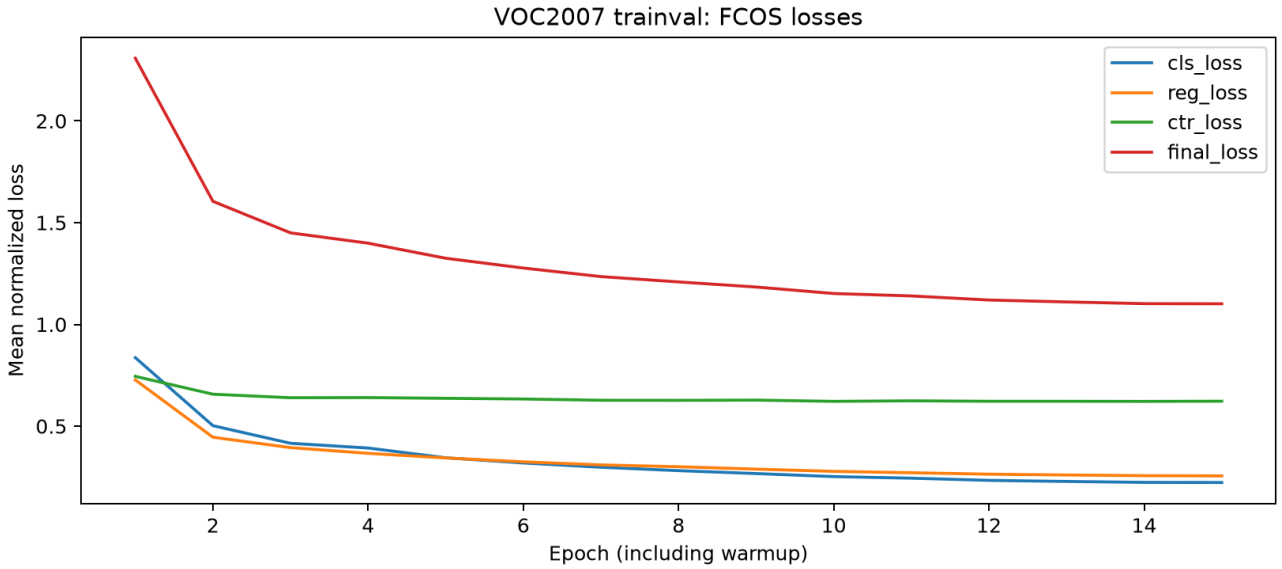



Figure 3. Recorded normalized losses over all 15 epochs (3 warmup + 12 cosine).

Training loops total 31.47 minutes (about 2.10 minutes per epoch); the runner records 31.79 minutes for its training invocation. This uses 15 epochs rather than the assignment's 10-epoch timing example. Total normalized loss falls from 2.3081 to 1.1013. Classification falls from 0.8365 to 0.2231, regression from 0.7269 to 0.2555, and centerness from 0.7447 to 0.6227. Continuous centerness targets admit a positive BCE floor. These curves show stable optimization, but one final test evaluation cannot establish overfitting or statistical significance. Test results did not select the checkpoint.

# 7. Instructor-reference detections

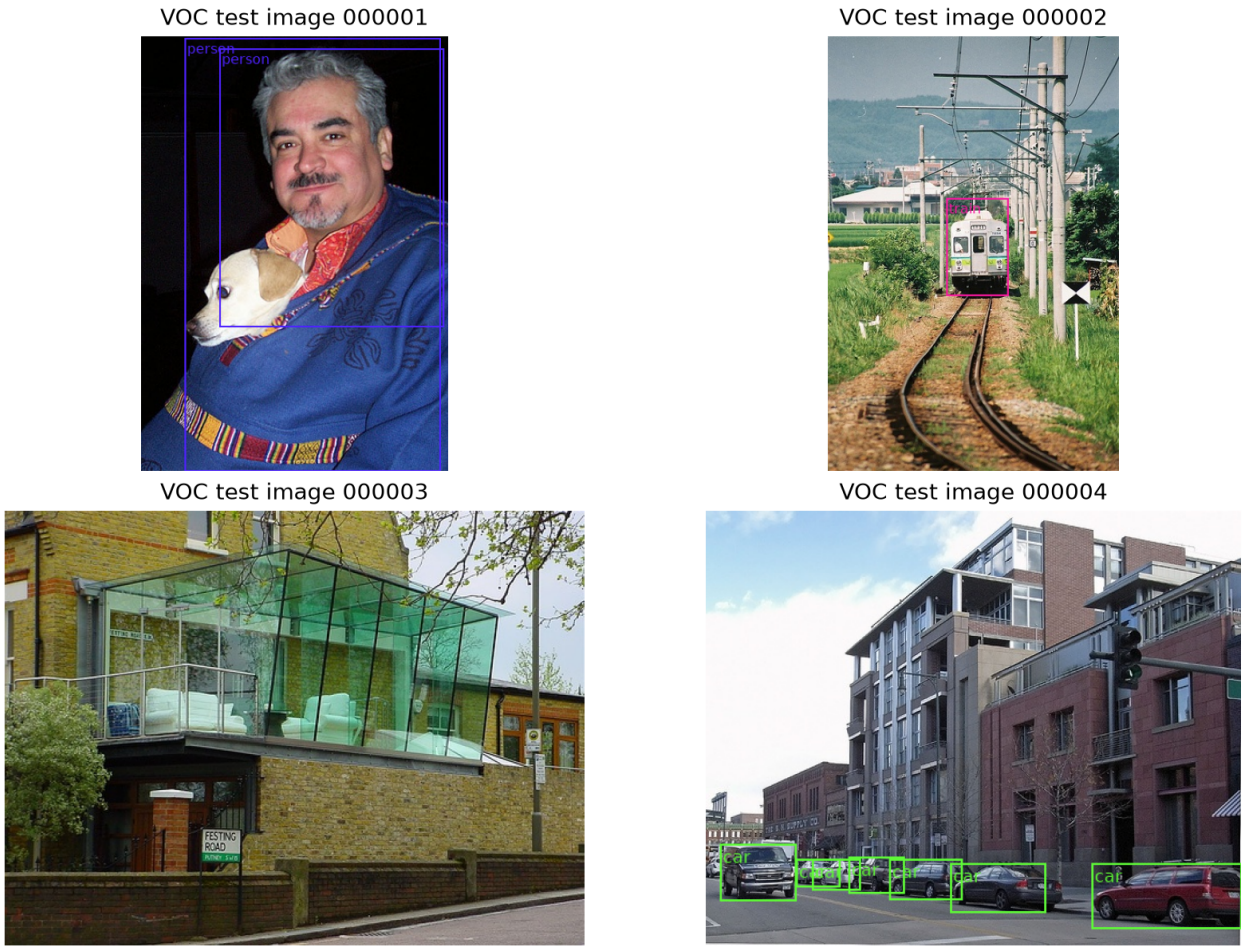



Figure 4. First four test images in deterministic loader order. Display threshold 0.5; AP evaluation threshold 0.1. The complete artifact set retains 12 sample images.

Image 000001 contains occlusion and overlapping hypotheses. The reference displays two person boxes with scores 0.6878 and 0.5277. Its highest dog candidate scores 0.4997, just below the display threshold. An absent displayed box can therefore still contribute to AP evaluation.

In image 000002, the reference displays a train at 0.5425. Image 000003 contains furniture behind glass and has no displayed prediction above 0.5. In image 000004, seven car boxes remain above the display threshold, including overlapping hypotheses among distant vehicles. NMS IoU 0.6 permits boxes whose overlap does not exceed that threshold.

These are fixed-order examples, rather than samples selected for success. The instructor checkpoint is used only as a correctness reference and is clearly separated from the independently trained detector.

# 8. Independently trained FCOS detections




Figure 5. Same four images and display settings as Figure 4, using the final trained FCOS checkpoint.

In image 000001, the trained detector displays one person at 0.7033. Its highest dog candidate is 0.4523, so it also falls below the display threshold. In image 000002, the highest train candidate is 0.4893: absent from the figure but retained at the 0.1 evaluation threshold. The figure alone would overstate this difference from the reference.

Neither detector displays a prediction above 0.5 in image 000003. Reflections, occlusion and small objects remain difficult in this example. In image 000004, the trained detector displays six car boxes versus seven for the reference. Similar aggregate AP50 can coexist with different confidence values and individual detections.

The qualitative examples help explain threshold and localization behavior; four examples cannot establish a general improvement or degradation. The full-test comparison remains 60.52% versus 60.69% AP50.

# 9. Verification, reproducibility and sources

All six submitted checkpoint/configuration pairs load strictly and contain finite tensors. Every trained model reaches its final scheduled epoch, with complete histories and nonempty TensorBoard events. The trained FCOS differs from the instructor model in 146 floating tensors. The transferred component matches all 144 recorded manifest hashes; original checkpoint and prediction digests are retained in the review evidence.

All four classifiers were independently re-evaluated on all 10,000 validation images, exactly reproducing top-1 and top-5. A second full FCOS inference on MPS with the saved batch size 32 yields 60.5192% AP50, differing from its submitted CUDA result by +0.000028 percentage points. The instructor MPS re-evaluation yields 60.6862%. Recomputing AP/AP50/AP75 from both original CUDA prediction files exactly reproduces all reported detector metrics. A separate batch-8 check yields 60.3142% AP50; changing batch composition changes spatial padding and GroupNorm outputs. Reproduction must preserve the saved evaluation batch size.

All 50 final automated tests pass. They check attention values/gradients, padded windows and stochastic depth, classifier outputs and parameter coverage, rectangular-grid point ordering, stride-aware box decoding, labels, clipping, class-aware NMS and empty predictions, target assignment, analytic focal/GIoU/centerness loss, empty-target backward passes, full FCOS gradients, stale audit rejection and clean archive selection. Detailed test output is retained under results/matej_review/.

The archive contains only code/, writeup/ and results/, with final weights, relative-path YAML files and logs. Datasets, environments, intermediate checkpoints and large prediction dumps are excluded. Obtain the original assignment data/ annotations and image datasets before rerunning evaluation. From code/, run eval_cls.py with the saved YAML and result directory, or eval_det.py with the YAML and final checkpoint. code/README.txt gives exact commands. code/package_submission.py checks and builds the archive; make_final_report.py regenerates this report from the audit evidence.

Additional common packages are pytest (tests), matplotlib (plots), reportlab (PDF generation) and pypdf (PDF inspection). Runtime dependencies are in code/requirements.txt; exact local/cloud packages are recorded under results/. Seeded procedures do not guarantee bitwise equality across devices, versions or resumed data-loader states. Historical logs preserve the actual runs. Original starter and third-party attributions remain intact.

# References


[1] BMI / CS 771 Fall 2026 Assignment 2 release: requirements, starter code, MiniPlaces/VOC annotations and instructor checkpoint.

[2] He et al. Deep Residual Learning for Image Recognition. CVPR, 2016. https://arxiv.org/abs/1512.03385

[3] Vaswani et al. Attention Is All You Need. NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[4] Dosovitskiy et al. An Image is Worth 16x16 Words. ICLR, 2021. https://arxiv.org/abs/2010.11929

[5] Xiong et al. On Layer Normalization in the Transformer Architecture. ICML, 2020. https://arxiv.org/abs/2002.04745

[6] Huang et al. Deep Networks with Stochastic Depth. ECCV, 2016. https://arxiv.org/abs/1603.09382

[7] Tian et al. FCOS: A simple and strong anchor-free object detector. Journal-version manuscript, 2020. https://arxiv.org/abs/2006.09214